Deep Learning for Friction-Induced Vibration

Corrected modelling, robust design, neural surrogates, neural operators and friction-law discovery

Vikash Kumar

Department of Mechanical Engineering

Indian Institute of Technology Kanpur

3 November 2026

Abstract

This project revisits my own earlier notebook on friction-induced vibration (published on GitHub in August 2024), corrects it, and tests where deep learning genuinely helps. The 2024 code injected energy through a sign error, had no driving belt, and optimised with finite-difference gradients that were pure solver noise. A corrected mass-on-moving-belt model was implemented as a batched, differentiable PyTorch simulator and validated against SciPy's Radau solver (relative error 1.1×10^{-5}). Four deep-learning components were built on it. (i) Robust damper design by stochastic gradient descent through the simulator cuts sustained stick-slip on unseen operating conditions from 93.3% to 3.0%. (ii) An MLP surrogate predicts the vibration level with $R^2 = 0.999$ and the stick-slip regime with 99.2% accuracy, against 97.1% for linear stability theory; the gap is a confirmed bistable band. (iii) A Fourier neural operator maps a time-varying belt speed to the full response with a 6.8% median velocity error. (iv) A hybrid neural ODE learns an unknown friction law from noisy synthetic vibration records; it recovers the law with an RMSE of 0.0041 in μ, seven times better than a fitted exponential law, and predicts the correct regime at unseen speeds. An independent code review found ten methodological problems in a first version, including validation leakage; all were fixed before the reported runs. The main remaining limitations are synthetic data, weakly identified static friction and an ensemble spread that is not a calibrated confidence interval.

Contents

1 Introduction and objectives **3**

2 Review of the 2024 implementation **3**

3 Corrected physical model and differentiable simulator **5**

4 Robust design by stochastic gradient descent with gradient clipping **5**

5 Neural surrogate and stick-slip classifier (MLP) **6**

6 Fourier neural operator for a time-varying belt speed **8**

7 Friction-law discovery with a hybrid neural-ODE ensemble: method **9**

8 Friction-law discovery: results **11**

8.1 Data and training 11
8.2 The discovered friction law 12
8.3 Free rollouts at training and test speeds 14
8.4 Phase portraits 16
8.5 Amplitude, period and stability 16
8.6 Residuals 18

9 Verification, limitations and discussion **18**

10 Conclusions and future work **20**

A Reproducing the results on a PyTorch PC **21**

1 Introduction and objectives

Friction-induced vibration (stick-slip, brake squeal, chatter in slides and lead screws) arises when friction falls as sliding speed rises. That velocity weakening acts as negative damping, so steady sliding can lose stability and the system settles on a stick-slip limit cycle [2, 3]. The behaviour is strongly nonlinear: the friction law is rarely known accurately, stick/slip switching is non-smooth, and more than one steady state can coexist.

The project started from my own earlier implementation, *Optimization of Friction-Induced Vibration System Using Stochastic Gradient Descent*, published on GitHub in August 2024 [1], which tunes damping, stiffness and friction parameters of a one-degree-of-freedom oscillator. A critical review of that code (Section 2) found that both its physics and its optimiser are incorrect. The model was therefore rebuilt, and the question became where deep learning genuinely helps. The objectives were:

1. correct the physical model and verify the numerical solver against an independent reference;
2. replace the finite-difference "SGD" with true stochastic gradient optimisation using exact gradients from a differentiable simulator;
3. build a neural surrogate and stick-slip classifier, and compare it with linear stability theory;
4. test a neural operator on a genuinely function-to-function problem, a time-varying sliding speed;
5. discover an unknown friction law from noisy vibration records with a hybrid physics + neural-network model, with an honest validation design and an uncertainty check.

The results come from two programs: `friction_vibration_dl.ipynb` (Sections 3–6) and `friction_law_discovery.py` (Sections 7–8); both run on a standard PyTorch installation (Appendix A). Every result in Sections 3–8 is printed by those programs; Section 2 uses a separate reproduction of the 2024 code. The friction-law study uses synthetic data from a hidden true law, so its accuracy can be measured directly.

2 Review of the 2024 implementation

The 2024 notebook has seven defects. The first two make the model non-physical, and the fourth turns its optimiser into a random walk. Each was reproduced by re-running the 2024 code (Table 1, Fig. 1).

Table 1: Defects of the 2024 implementation.

#	Defect	Evidence	Fix in this project
1	Friction written as $+F\,\mathrm{sign}(\dot{x})$, i.e. along the velocity	Friction power is never negative; mechanical energy rises from 0.50 J to about 1.06 J with no energy source (Fig. 1a,b)	Friction opposes the slip velocity $v_r = v_b - \dot{x}$
2	No moving belt or disc	Nothing can drive self-excited vibration; the only cycle is created by the sign error	Mass on a moving belt, the standard stick-slip model
3	Objective $\max E - \min E$ of mechanical energy	Not a vibration measure; the Bayesian search ends with $k = 20$ and $\alpha = 1.0$, both on their bounds ($c = 0.389$)	Late-time vibration RMS plus a cost on damping
4	"SGD" uses forward differences, $h = 10^{-5}$, through an adaptive solver at default tolerance	Computed gradient at the start point $(-738, -5272, -7185, -11697)$; true gradient $(-2.75, 0.02, 14.3, -1.49)$. After the first step the loss stays at 0.659 for the rest of the 10 000 iterations	Exact gradients by automatic differentiation through a fixed-step solver
5	Cells described as SGD actually call `gp_minimize` (Bayesian optimisation)	Method mislabelled in README and code	Adam with gradient-norm clipping
6	R^2 computed between a prediction and itself	Prints $R^2 = 1.0$ in both cells, whatever the parameters	R^2 of surrogates on held-out test data
7	$\mathrm{sign}(\dot{x})$ discontinuity, unbounded parameters	Solver chatter; parameters can leave the physical range	$\tanh(v_r/\varepsilon)$ regularisation; sigmoid or log re-parameterisation

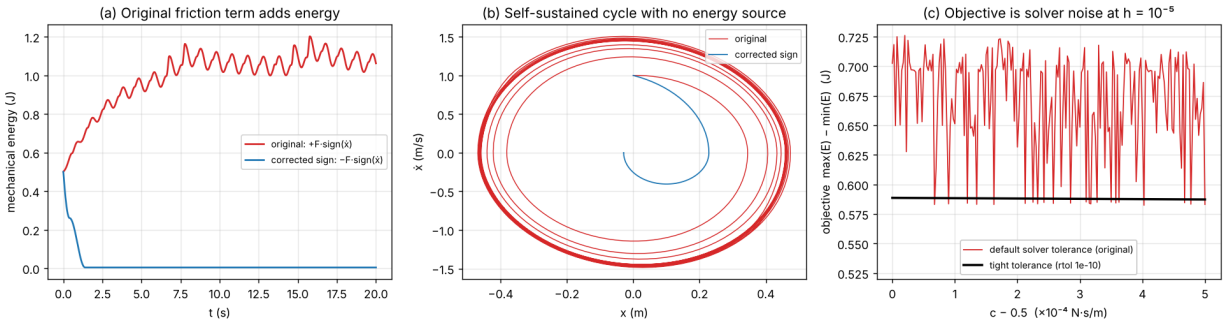


Figure 1: The 2024 model reproduced. (a) Energy rises because the friction term pushes the mass along its velocity. (b) The resulting self-sustained cycle (red) against the same model with the sign corrected (blue), which comes to rest. (c) The original objective as c changes by only 5×10^{-4}: at the default solver tolerance (red) it jumps by up to 0.14 J, while the converged value (black) is flat. A finite difference with $h = 10^{-5}$ therefore measures solver noise, not the gradient; the error is two to five orders of magnitude.

3 Corrected physical model and differentiable simulator

The corrected model is the classical mass on a moving belt:

$$m\ddot{x} + c\dot{x} + kx = N\,\mu(v_r)\,\tanh\!\left(\frac{v_r}{\varepsilon}\right), \qquad v_r = v_b - \dot{x}, \qquad N = mg, \tag{1}$$

$$\mu(v_r) = \mu_k + (\mu_s - \mu_k)\,e^{-\alpha|v_r|}. \tag{2}$$

Here x is the displacement of the mass, v_b the belt speed and v_r the slip speed. The sign function of Coulomb friction is replaced by $\tanh(v_r/\varepsilon)$ with $\varepsilon = 0.02\,\mathrm{m/s}$, which keeps the right-hand side smooth and differentiable. Sections 3–6 use $m = 1\,\mathrm{kg}$ and $N = 9.81\,\mathrm{N}$.

Linear stability of steady sliding. In steady sliding the mass rests at $x = N\mu(v_b)/k$. Linearising the friction force about that state adds $N\mu'(v_b)$ to the damping, so steady sliding is unstable when the total damping is negative:

$$c < c_{\mathrm{crit}} = -N\,\mu'(v_b) = N(\mu_s - \mu_k)\,\alpha\,e^{-\alpha v_b}. \tag{3}$$

For $v_b \gg \varepsilon$ the tanh term adds a negligible $N\mu\,\mathrm{sech}^2(v_b/\varepsilon)/\varepsilon$. This analytic boundary is the reference for the learned models throughout.

Simulator. A batched, fixed-step fourth-order Runge–Kutta integrator written in PyTorch [11] (float64, $\Delta t = 4\,\mathrm{ms}$). Every operation is differentiable, so gradients of any output with respect to c, k or network weights come from automatic differentiation, and thousands of parameter sets are integrated at once as one tensor.

Validation. The same smoothed ODE was solved with SciPy's implicit Radau method at rtol $= 10^{-9}$ for $c = 0.3\,\mathrm{N\,s/m}$, $k = 20\,\mathrm{N/m}$, $v_b = 0.4\,\mathrm{m/s}$, $\mu_s = 0.55$, $\mu_k = 0.30$, $\alpha = 2$. The maximum relative displacement error of the RK4 solver over 20 s is 1.08×10^{-5}. For these values $c_{\mathrm{crit}} = 2.204\,\mathrm{N\,s/m}$, far above $c = 0.3$, so stick-slip is expected, as Fig. 2 shows.

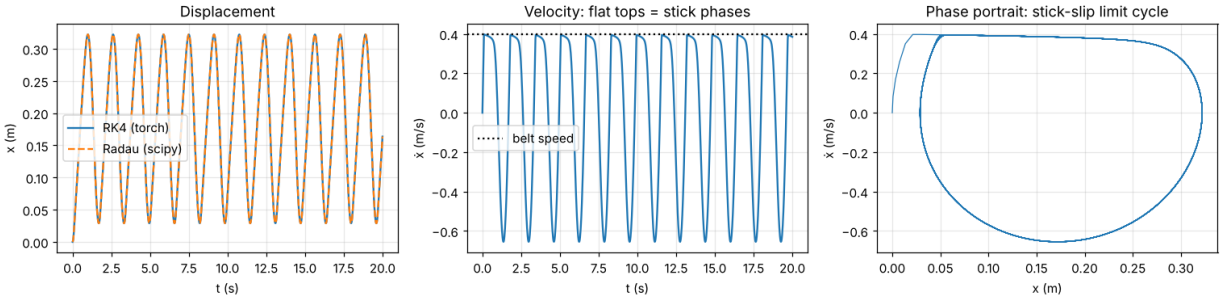


Figure 2: Left: RK4 (PyTorch) and Radau (SciPy) displacements overlap. Centre: the velocity has flat tops at the belt speed (0.4 m/s); these are the stick phases. Right: the stick-slip limit cycle in the phase plane.

4 Robust design by stochastic gradient descent with gradient clipping

True stochastic gradient descent through the simulator raises the damping from 0.77 to 2.09 N s/m and cuts the mean vibration level from 0.449 to 0.013 m/s over 400 unseen operating conditions.

Formulation. The design variable is the damping $c \in [0.05, 4]\,\mathrm{N\,s/m}$, mapped from an unconstrained parameter θ by a sigmoid; $k = 20\,\mathrm{N/m}$. The operating conditions are uncertain and drawn afresh at every step: $v_b \sim U(0.3, 1.0)\,\mathrm{m/s}$, $\alpha \sim U(1, 2)\,\mathrm{s/m}$, $\mu_s \sim U(0.45, 0.55)$, $\mu_k \sim U(0.28, 0.32)$. The objective trades vibration against the cost of damping:

$$J(c) = \mathbb{E}_{v_b,\alpha,\mu_s,\mu_k}\left[\mathrm{RMS}(\dot{x})_{12\text{–}20\,\mathrm{s}}\right] + \lambda c, \qquad \lambda = 0.05\ \frac{\mathrm{m/s}}{\mathrm{N\,s/m}}. \tag{4}$$

Each step simulates 24 random conditions for 20 s from rest, back-propagates through all 5 000 RK4 steps, clips the gradient norm at 1 and takes an Adam step (learning rate 0.08, 80 steps, 495 s on a CPU). Because each step sees different random conditions, its gradient is a noisy but unbiased estimate of $\mathrm{d}J/\mathrm{d}c$; this is what makes the method stochastic gradient descent.

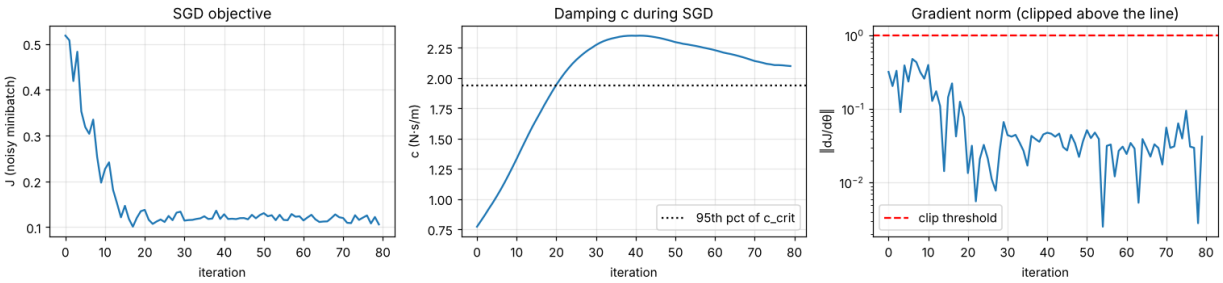


Figure 3: Left: the noisy minibatch objective falls from 0.52 to about 0.12 within 25 steps. Centre: c rises past the 95th percentile of the analytic c_{crit} (dotted) and settles near 2.1 N s/m. Right: the gradient norm stays below the clipping threshold of 1 throughout this run, so clipping acted only as a safeguard.

Check on unseen conditions. Both designs were simulated on 400 new random conditions (Table 2). "Sustained" means the vibration is still above 0.01 m/s RMS after 200 s.

Table 2: Start and optimised designs on 400 unseen operating conditions.

Design	c (N s/m)	J	Mean RMS($\dot{x}$), 12–20 s (m/s)	RMS > 0.01 at 12–20 s	Sustained after 200 s	$c < c_{\mathrm{crit}}$ (linear theory)
Start	0.771	0.4877	0.4492	100.0%	93.3%	80.5%
SGD optimum	2.094	0.1179	0.0132	13.2%	3.0%	2.8%

- The optimum sits just above the bulk of the stability boundary: c_{crit} over the 400 conditions has a 95th percentile of 1.939 and a maximum of 2.372 N s/m (worst case of the whole uncertainty box: 2.907). Covering the last few per cent of rare conditions costs more damping than it saves vibration at $\lambda = 0.05$.
- At the start design 93.3% of conditions sustain stick-slip, although linear theory predicts only 80.5%. The difference is bistability, analysed in Section 5.
- The objective is flat in k: at $c = 2.094$, $J = 0.1184,\ 0.1179,\ 0.1178,\ 0.1177$ for $k = 10,\ 20,\ 30,\ 50\,\mathrm{N/m}$. This is why k was removed from the design variables; the stability limit (3) contains no k.

5 Neural surrogate and stick-slip classifier (MLP)

A small multilayer perceptron predicts the vibration level with $R^2 = 0.999$ (log scale) and classifies sustained stick-slip with 99.2% accuracy on unseen cases. That beats linear stability theory (97.1%),

because the network also captures bistability.

Why an MLP. The inputs are six scalars $(c, k, v_b, \alpha, \mu_s, \mu_k)$ and the outputs are a number and a class. A plain MLP is the right tool; a neural operator adds nothing when neither input nor output is a function.

Data. 8 000 random parameter sets drawn uniformly from $c \in [0.05, 4]$, $k \in [5, 50]$, $v_b \in [0.1, 1.5]$, $\alpha \in [0.5, 3]$, $\mu_s \in [0.40, 0.65]$, $\mu_k \in [0.20, 0.35]$, each simulated from rest by the batched solver (66 s in total), split 6 000/1 000/1 000 for training, validation and test.

- *Regression target:* $\log_{10}(\mathrm{RMS}(\dot{x}) + 10^{-4})$ over 12–20 s, the same measure as in J.
- *Class label:* sustained stick-slip, $\mathrm{RMS}(\dot{x}) > 0.01$ m/s over 120–200 s of a 200 s run (32.5% of samples). Re-running the test set for 400 s changes only 2 of 1 000 labels, so the labels are settled.

Network. Three hidden layers of 128 SiLU units with two heads; loss = MSE + 0.3× binary cross-entropy; AdamW, cosine schedule, 600 epochs; the checkpoint with the lowest validation loss is restored at the end.

Table 3: Test metrics on 1 000 unseen cases.

Metric	Value
R^2 of $\log_{10}$ vibration level	0.9989
R^2 of $\mathrm{RMS}(\dot{x})$ in m/s	0.9858
Classifier accuracy	99.2%
Accuracy of the linear rule $c < c_{\mathrm{crit}}$	97.1%

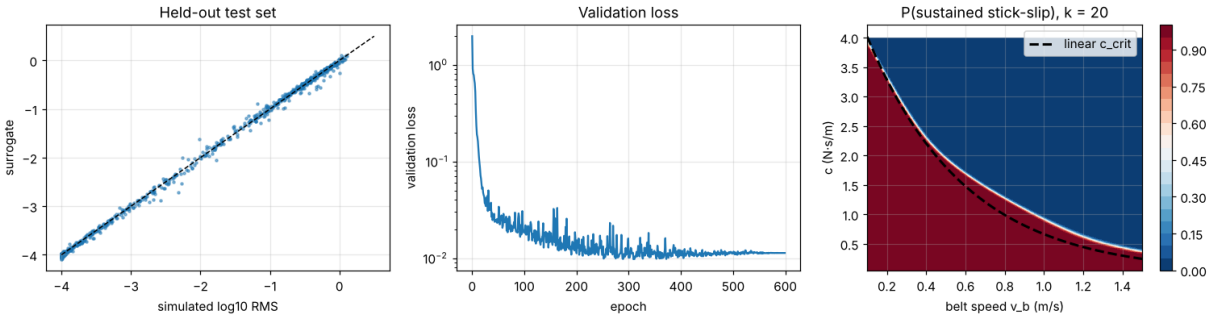


Figure 4: Left: surrogate against simulator on the test set. Centre: validation loss. Right: classifier probability of sustained stick-slip over belt speed and damping ($k = 20$, $\mu_s = 0.55$, $\mu_k = 0.30$, $\alpha = 2$); the dashed line is the analytic c_{crit}. The learned boundary lies above c_{crit} at all but the lowest speeds, where both reach the top of the c range.

Bistability. Of the 344 test cases with sustained stick-slip, 29 have $c > c_{\mathrm{crit}}$, so steady sliding is linearly stable there. Each of those 29 was re-run starting next to steady sliding (1 mm from equilibrium, at rest): all 29 settled to steady sliding. Steady sliding and a stick-slip cycle therefore coexist, and which one is reached depends on how the system is disturbed. This band between c_{crit} and the learned boundary is what the classifier captures and linear theory cannot; a damper sized only by linear stability is therefore non-conservative.

Design through the surrogate. Repeating the Section 4 optimisation with the surrogate in place of the simulator (4 096 conditions per step, 400 steps) takes 4.2 s instead of 495 s. Checked with the true simulator on the same 400 unseen conditions (Table 4), the surrogate design is within 0.1% of the simulator-based design in J. Each optimisation is about 120 times faster once the surrogate exists; its data and training are a one-off cost.

Table 4: Designs obtained through the simulator and through the surrogate, both checked with the true simulator.

Design	c (N s/m)	J	Mean RMS($\dot{x}$) (m/s)	Sustained after 200 s
SGD through simulator (Sec. 4)	2.094	0.1179	0.0132	3.0%
SGD through surrogate	2.081	0.1180	0.0140	3.5%

6 Fourier neural operator for a time-varying belt speed

A one-dimensional Fourier neural operator (FNO) [8] maps a whole belt-speed history to the whole 10 s response with a median velocity error of 6.8% on unseen profiles. For this cheap 1-DOF model it is only about 1.5 times faster than the batched simulator, so its value here is as a tested method for more expensive models.

Why an operator here. In a braking event or a variable-speed drive the sliding speed changes during the test, so the input is a function, not a number. The task is to learn the map

$$\mathcal{G}: \ (v_b(t),\, c) \ \mapsto \ (x(t),\, \dot{x}(t)), \qquad t \in [0, 10] \text{ s}. \tag{5}$$

Data. 2 400 random speed profiles: a mean of 0.3–1.0 m/s, a linear trend of up to ± 0.6 m/s (for example a braking ramp) and four random harmonics, clipped to 0.05–1.6 m/s. Damping c is drawn from $[0.2, 3]$ N s/m; friction is the nominal exponential law ($\mu_s = 0.55$, $\mu_k = 0.30$, $\alpha = 2$) with $k = 20$ N/m. Responses are stored at 25 Hz (251 points). 2 000 profiles train the model and 400 are held out; output scaling uses the training set only.

Architecture. Inputs $v_b(t)$, c and t as three channels $\rightarrow$ lift to 48 channels $\rightarrow$ four Fourier layers that keep 48 modes, each a spectral convolution plus a pointwise convolution with GELU $\rightarrow$ projection to $x(t)$ and $\dot{x}(t)$. 458k parameters; AdamW with a one-cycle schedule, 150 epochs (659 s on a CPU).

Table 5: FNO test errors on 400 unseen profiles (relative L_2).

Quantity	Median	90th percentile
$x(t)$	2.4%	10.1%
$x(t)$ minus its time mean	6.2%	25.9%
$\dot{x}(t)$	6.8%	27.1%

The first row flatters the model: $x(t)$ is dominated by the quasi-static deflection $N\mu/k \approx 0.2$ m, so the second row, the vibration part alone, is the honest displacement measure. The vibration level predicted from the FNO output (RMS of $\dot{x}$ over 5–10 s) has $R^2 = 0.987$ against the simulator. The FNO produced the 400 test responses in 339 ms (float32) against 512 ms for the batched RK4 simulator (float64).

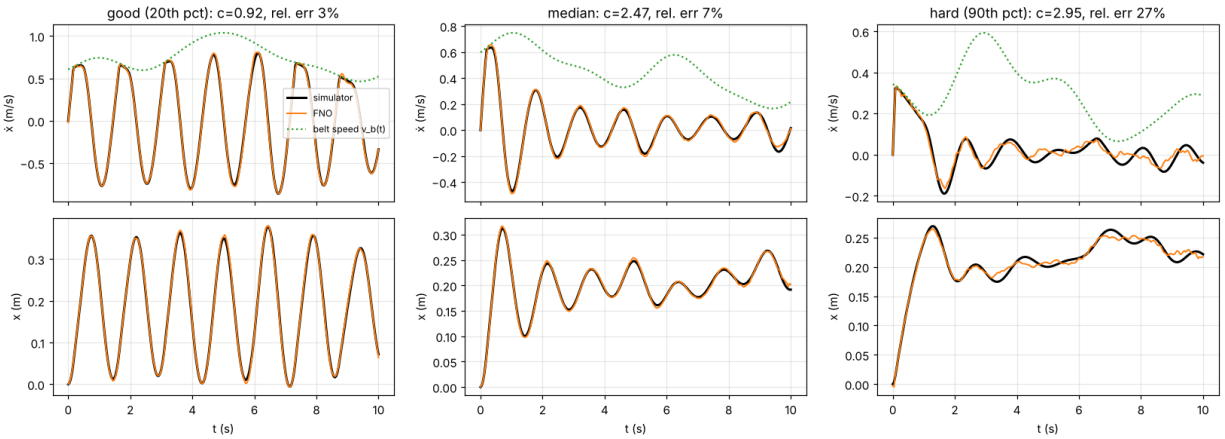


Figure 5: FNO (orange) against the simulator (black) for test profiles at the 20th, 50th and 90th percentile of velocity error; the dotted green line is the belt-speed input. The hard case (right) has a heavily damped, low-amplitude response while the belt slows towards stick speeds: a small absolute error is a large relative error, and the response switches between near-stick and sliding.

Where it fails and why. The largest errors occur when the response switches regime within the record. That switching is the sharpest feature of the operator, and 2 000 random profiles cover it thinly. Remedies are more training profiles near the boundary (the simulator is cheap), an architecture suited to non-smooth switching (a U-Net or transformer operator), or the hybrid model of Sections 7–8, which keeps the switching inside the physics.

7 Friction-law discovery with a hybrid neural-ODE ensemble: method

The mechanics stay physics and only the friction coefficient is a neural network [5, 6], so the learned model is always physically admissible: friction opposes slip, is odd in v_r, and $\mu \geq 0$.

$$m\ddot{x} + c\dot{x} + kx = N\,\mu_\theta(|v_r|)\,\tanh\!\left(\frac{v_r}{\varepsilon}\right), \qquad \mu_\theta = \mathrm{softplus}\big(\mathrm{MLP}_\theta(|v_r|/v_{\mathrm{scale}})\big). \tag{6}$$

Network. A $1 \to 32 \to 32 \to 1$ MLP with tanh activations and a softplus output, initialised near $\mu = 0.35$. The input is scaled by the 99.5th percentile of the slip speeds seen in training.

Multiple shooting. Each record is cut into 0.5 s windows (25 samples at 50 Hz, a new window every 5 samples) [7]. Every window starts from the measured state and is integrated with the differentiable RK4 solver (8 sub-steps per sample, $\Delta t = 2.5$ ms). Short windows avoid back-propagating through long, nearly chaotic stick-slip histories, which is what usually makes neural-ODE training fail on such systems. The loss is the mean squared error in x and $\dot{x}$ over samples 1–25 of each window, each divided by its measurement-noise variance, so a perfect model on pure white noise would score 2.

Noise estimate. σ is the robust (median-absolute-deviation) spread of the residual from a 5-point cubic Savitzky–Golay smoothing [10], multiplied by $1/\sqrt{1-h_0} = 1.39$, where $h_0 = 17/35$ is the filter's centre weight. Without that factor the filter's own pass-through of noise makes the estimate 28% too low.

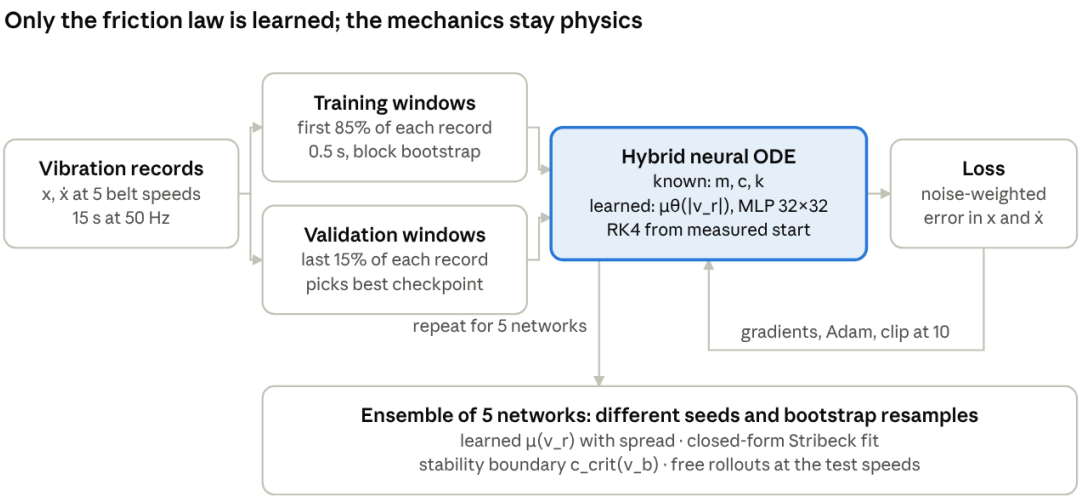


Figure 6: Training pipeline of the hybrid neural ODE. Each training window is integrated from its own measured start state; the error against the measurement trains only the friction network, and five such networks form the ensemble.

Validation design. The last 15% of every training record (in time) is held out; training windows come only from the first 85%, so no validation sample is ever seen in training. Separate test experiments at other belt speeds are used only for the final free rollouts.

Ensemble. Five networks [9], each trained on a block-bootstrap resample of the training windows (1.5 s blocks, so overlapping windows are resampled together) from a different random start. Adam, learning rate 3×10^{-3} with cosine decay, 600 iterations of 128 windows, gradient norm clipped at 10; the checkpoint with the lowest validation loss is kept. The spread of the five curves is reported as an ensemble spread, not as a calibrated confidence interval; its actual coverage is measured in Section 8.

Baseline. The classical exponential law (2) with the same three-parameter fit through the same simulator, loss, validation and iteration budget.

Synthetic experiment. The data come from a law the model is not told:

$$\mu_{\text{true}}(v) = 0.25 + 0.20\, e^{-(|v|/0.25)^2} + 0.08\,|v|. \tag{7}$$

Table 6: Synthetic rig used for the friction-discovery demo.

Item	Value
Rig	$m = 1\,\text{kg}$, $c = 0.2\,\text{N s/m}$, $k = 20\,\text{N/m}$, $N = 9.81\,\text{N}$
Records	15 s each, sampled at 50 Hz, released from rest
Training belt speeds	0.15, 0.30, 0.50, 0.80, 1.20 m/s
Test belt speeds	0.10 (below the training range), 0.65, 1.00 m/s
Measurement noise	$\sigma_x = 2\,\text{mm}$, $\sigma_v = 0.01\,\text{m/s}$, white Gaussian
True system sharpness	$\varepsilon = 0.005\,\text{m/s}$, four times sharper than the model's 0.02, so the model is not the exact data generator

Post-processing.

- Closed-form fit of a generalised Stribeck curve $\mu_k + (\mu_s - \mu_k)\exp(-(|v|/v_s)^\delta) + \beta|v|$ to the ensemble mean, and to each member to show the parameter spread.
- A direct inverse-dynamics estimate $\mu = (m\ddot{x} + c\dot{x} + kx)/(N \operatorname{sign} v_r)$ from smoothed signals, an independent check that needs m, c, k but no training.
- The linear stability limit of steady sliding for the model actually simulated, including the tanh term: $c_{\text{crit}} = -\,\mathrm{d}F/\mathrm{d}v_r$ at $v_r = v_b$, with $F = N\mu(v)\tanh(v/\varepsilon)$.
- Vibration amplitude from the last third of each record after light smoothing. A case counts as sustained stick-slip only if the amplitude exceeds $5\sigma_x$ and is not still decaying (at least 0.7 of the middle third). Periods use crossings with hysteresis over the last half, for sustained cases only.
- Below $|v_r| = \max(5\varepsilon,\, 10\sigma_v) = 0.10\,\mathrm{m/s}$ the slip speed during stick and creep is buried in velocity noise; that region is shaded as weakly identified in every plot.

8 Friction-law discovery: results

Where the data constrain it (0.10–1.62 m/s), the hybrid model recovers the hidden friction law to an RMSE of 0.0041 in μ, seven times closer than the best-fit exponential law (0.0293). It is the only model that correctly predicts steady sliding at the unseen speed of 1.0 m/s. Its weak points are static friction below 0.1 m/s and an overconfident ensemble spread.

8.1 Data and training

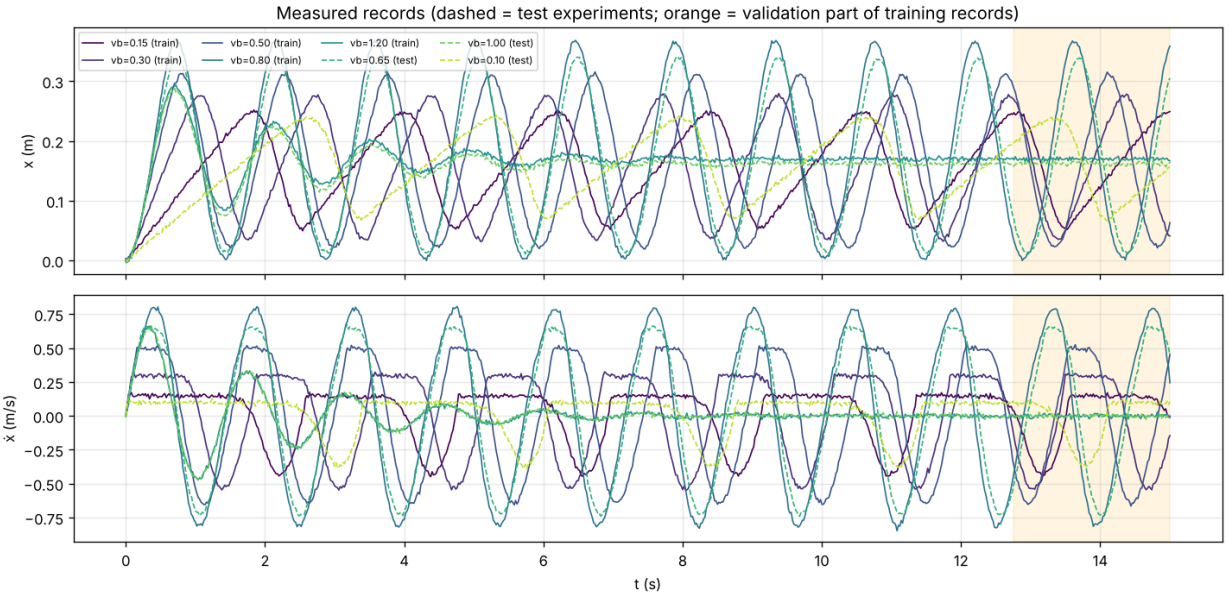


Figure 7: The eight synthetic "measured" records (x top, $\dot{x}$ bottom). Solid lines are training speeds, dashed lines test speeds. The orange band is the last 15% of each training record, held out for validation. Flat tops in $\dot{x}$ at the belt speed are stick phases; at 1.0 and 1.2 m/s the response decays to steady sliding.

The records give 615 training windows (in 45 bootstrap blocks) and 90 validation windows. The noise estimator recovers the true noise: $\sigma_x = 2.05\,\mathrm{mm}$ (true 2.00) and $\sigma_v = 0.0099\,\mathrm{m/s}$ (true 0.010).

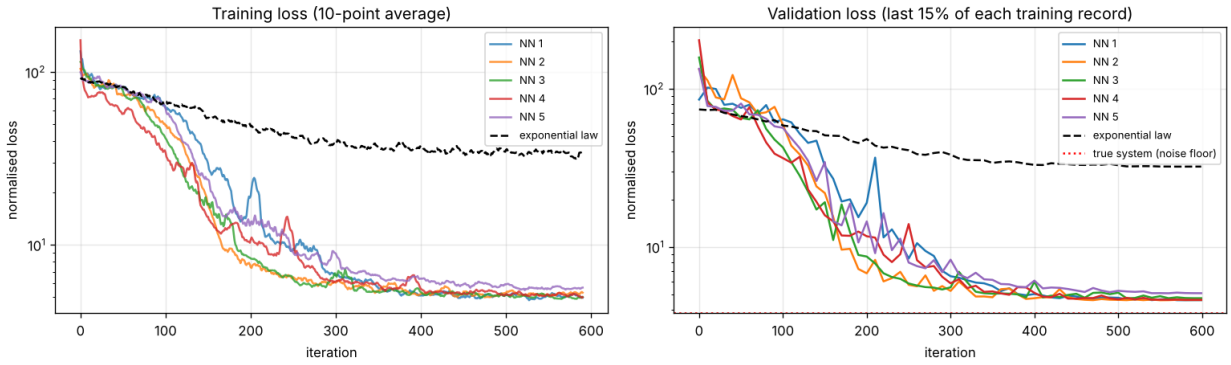


Figure 8: Training loss (left) and held-out validation loss (right) of the five networks and the exponential-law baseline. The red dotted line is the loss of the true system on the same windows.

Table 7: Best validation loss (noise-normalised).

Model	Best validation loss
True system (noise floor)	3.83
Hybrid neural ODE, five networks	4.60, 4.60, 4.73, 4.63, 5.09 (mean 4.73)
Exponential law	32.20

The floor is above the white-noise value of 2 because each window starts from a noisy measured state, and that initial error propagates through the window. The best networks come within 0.8 of the floor, and the five networks within 0.9 on average. The exponential law cannot: its shape does not allow a minimum followed by a rise in μ.

8.2 The discovered friction law

Table 8: Accuracy of the learned friction law against the hidden law.

Measure	Hybrid neural ODE	Exponential law
RMSE of μ, identified range 0.10–1.62 m/s	0.0041	0.0293
RMSE of μ, weak range below 0.10 m/s	0.0329	–
μ at zero slip (true 0.450)	0.514	0.502
Share of the identified range where the $\pm 2\sigma$ spread contains the truth	52%	–
Same, weak range	12%	–

The network reproduces the velocity-weakening drop, the minimum near 0.5 m/s and the viscous rise that the exponential law cannot represent. Two limitations are visible:

- **Static friction is overestimated** (0.514 against 0.450). During stick the slip speed is millimetres per second, below the 0.01 m/s velocity noise, so the data barely constrain μ there.
- **The ensemble spread is overconfident.** A calibrated $\pm 2\sigma$ interval would contain the truth about 95% of the time; this one does so on 52% of the identified range. The five networks see almost the same data, so their spread reflects optimiser scatter more than true uncertainty.

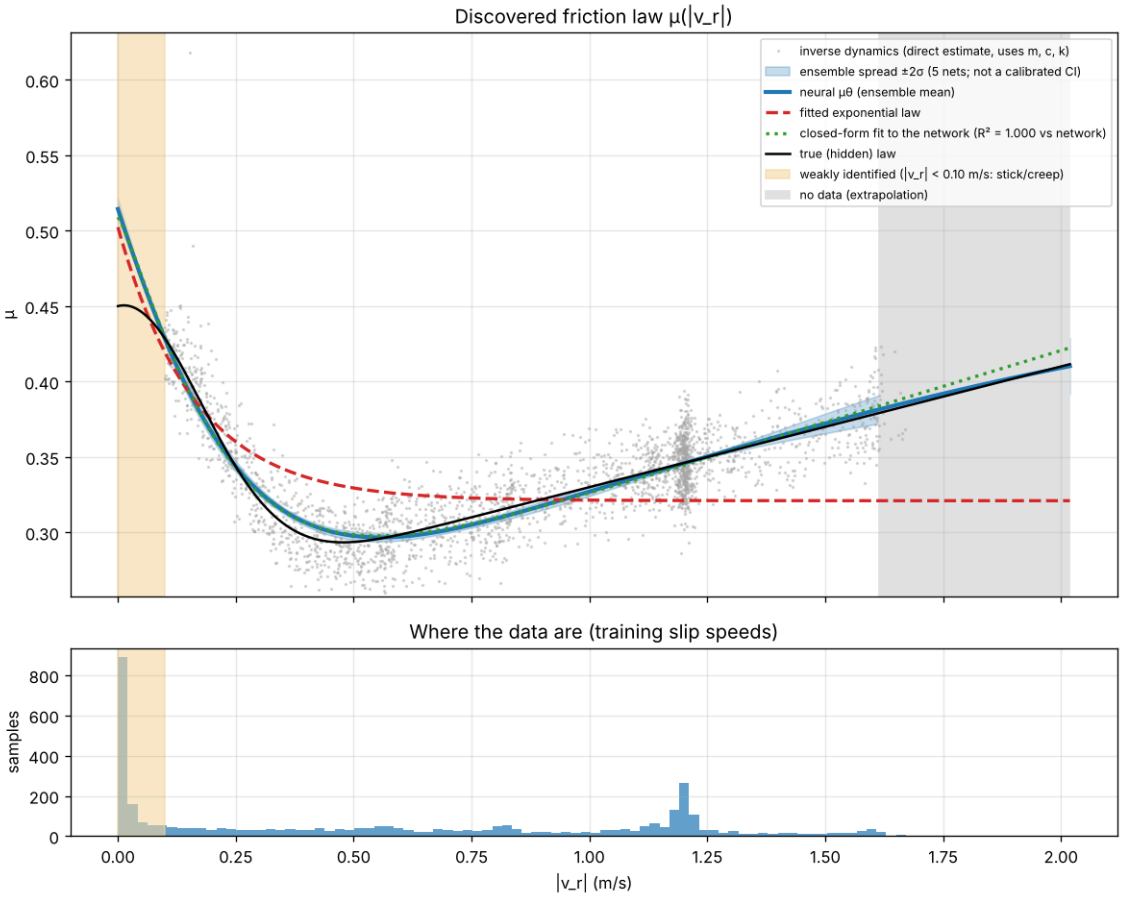


Figure 9: Top: ensemble-mean neural law (blue) with its $\pm 2\sigma$ spread, fitted exponential law (red dashed), closed-form fit to the network (green dotted) and the hidden true law (black). Grey dots are a direct inverse-dynamics estimate from the signals. The orange band ($|v_r| < 0.10\,\mathrm{m/s}$) is weakly identified; the grey band has no data. Bottom: number of training samples at each slip speed.

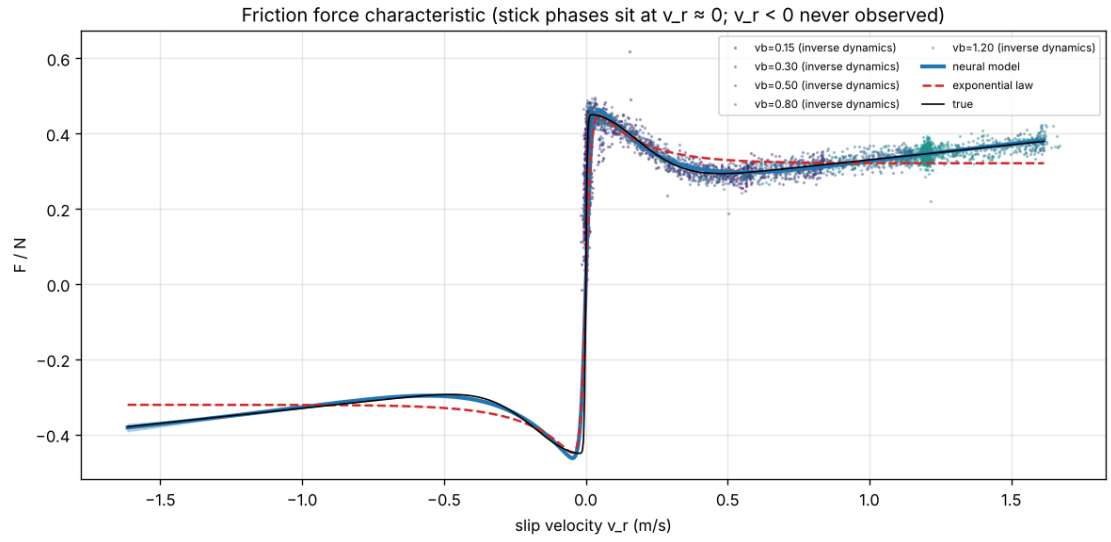
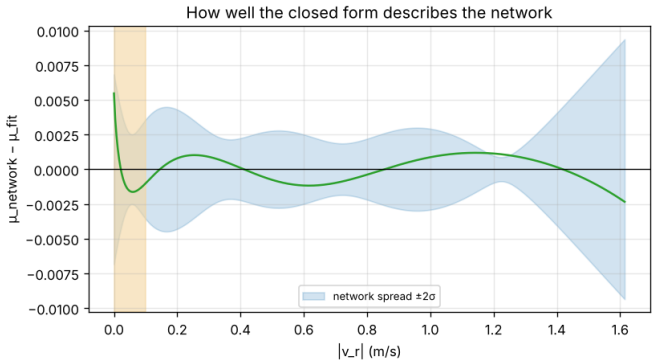


Figure 10: Friction force per unit normal load against slip velocity. The model's force is odd by construction; only $v_r > 0$ is observed, because the mass never moves faster than the belt.

Closed form. A generalised Stribeck curve describes the network almost perfectly ($R^2 = 0.9995$ against the network mean), giving a compact formula for the learned law. Its parameters should not, however, be read as identified physical constants (Table 9): μ_s inherits the weak-region error, and v_s, δ and β trade off against each other, so several parameter sets describe nearly the same curve. As with the band, the spread over networks understates the real error.

Table 9: Closed-form (generalised Stribeck) parameters fitted to the learned law.

Parameter	Fitted (± spread over 5 networks)	True
μ_s, static	0.509 ± 0.005	0.45
μ_k, kinetic	0.230 ± 0.010	0.25
v_s, Stribeck speed (m/s)	0.223 ± 0.010	0.25
δ, shape exponent	1.19 ± 0.08	2
β, viscous slope (s/m)	0.095 ± 0.008	0.08



μ = μ_k + (μ_s−μ_k)·exp(−(|v|/v_s)^δ) + β|v|

parameter	fitted (± spread over nets)	true
μ_s (static, weakly identified)	0.509 ± 0.005	0.45
μ_k (kinetic)	0.230 ± 0.010	0.25
v_s (Stribeck speed)	0.223 ± 0.010 m/s	0.25 m/s
δ (shape exponent)	1.185 ± 0.076	2
β (viscous)	0.095 ± 0.008 s/m	0.08 s/m
R^2 (fit vs network mean)	0.9995	—

Figure 11: Left: difference between the network mean and its closed-form fit; it stays below 0.006 in μ, about the size of the network spread. Right: fitted parameters.

8.3 Free rollouts at training and test speeds

Each model was released from the first measured sample and integrated for the full 15 s with no further use of the data. Errors therefore accumulate, which makes this a much harder test than the 0.5 s training windows.

Table 10: Vibration amplitude and period at all belt speeds. Amplitudes are half the peak-to-peak displacement over the last third of each record after light smoothing; "true" is the noise-free signal. The neural-ODE column is the mean over five networks, with the number of networks that sustain stick-slip in brackets. Periods are measured values (the noise-free truth agrees within 0.01 s), for sustained stick-slip only.

v_b (m/s)	Role	Behaviour (measured)	True amp. (mm)	Neural ODE mean of 5 (mm)	Exponential (mm)	Period (s) meas. / NODE / exp.
0.10	test, below range	stick-slip	84.3	81.4 (5/5)	57.0	2.68 / 2.86 / 2.32
0.15	train	stick-slip	97.3	95.4 (5/5)	72.7	2.19 / 2.27 / 1.96
0.30	train	stick-slip	120.0	119.4 (5/5)	105.0	1.67 / 1.68 / 1.60
0.50	train	stick-slip	144.3	142.8 (5/5)	142.2	1.48 / 1.48 / 1.48
0.65	test	stick-slip	162.8	160.0 (5/5)	170.0	1.44 / 1.44 / 1.44
0.80	train	stick-slip	181.7	112.7 (3/5)	198.4	1.43 / 1.43 / 1.43
1.00	test	decays to sliding	1.1	0.6 (0/5)	93.1, sustained	– / – / 1.42
1.20	train	decays to sliding	1.0	0.6 (0/5)	58.6, decaying	–

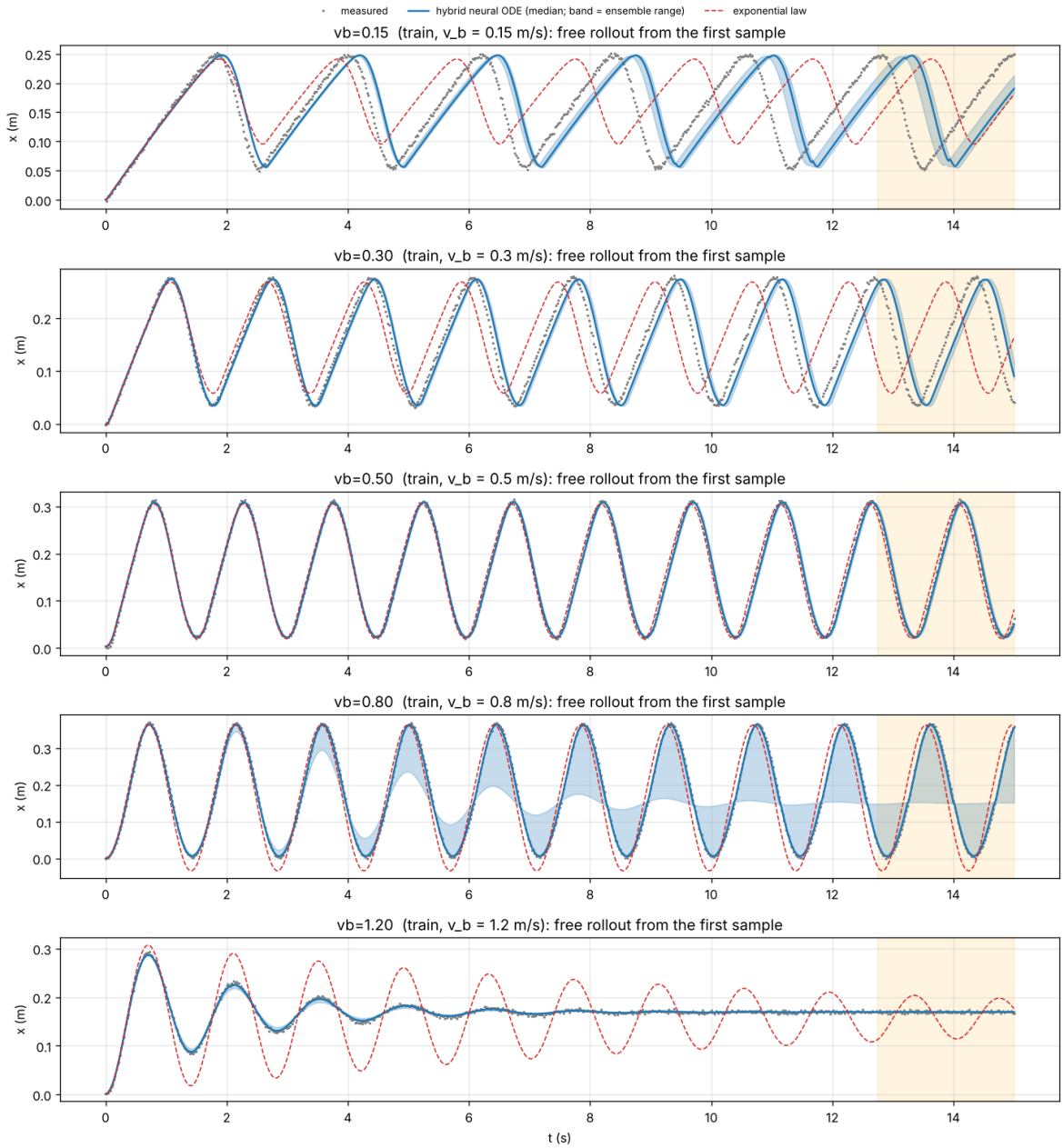


Figure 12: Free rollouts at the five training speeds: measured (grey dots), hybrid neural ODE (blue: median of the five networks; band: their range) and exponential law (red dashed). The orange band marks the held-out validation part.

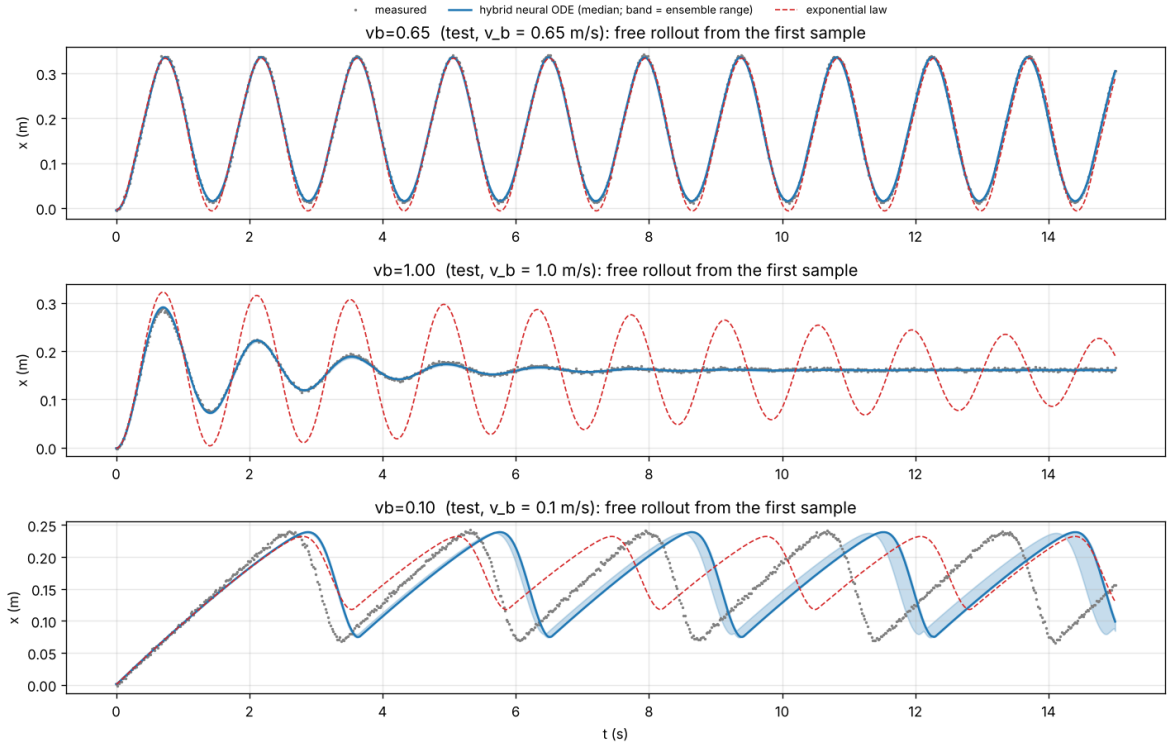


Figure 13: Free rollouts at the three test speeds, never used in training or model selection.

The test speeds separate the models clearly:

- **1.0 m/s (test):** the true system settles to steady sliding. All five networks predict this; the exponential law predicts sustained chatter of 93 mm, the wrong regime.
- **0.65 m/s (test):** both models are close (amplitude −1.7% for the network, +4.4% for the exponential law).
- **0.10 m/s (test, below every training speed):** the network amplitude is within 3.4%, but its period is 7% long (2.86 s against 2.68 s), so its cycle drifts out of phase (Fig. 13, bottom). The overestimated static friction lengthens each stick phase. The exponential law is worse in both amplitude (−32%) and period (−13%).
- **0.80 m/s (train):** only three of the five networks reproduce the stick-slip cycle; two settle to sliding (about 180 mm against 8 mm, hence the mean of 112.7 mm). Section 8.5 shows that this speed is bistable, where small differences in the law change which state is reached; the ensemble disagreement correctly signals low confidence.

8.4 Phase portraits

Figure 14 compares the limit cycles in the phase plane. The network reproduces the measured cycle shape, including the flat stick phase, at every speed where stick-slip is sustained; the exponential law gives cycles that are too small at low speed.

8.5 Amplitude, period and stability

For $v_b \geq 0.10$ m/s the network's c_{crit} has an RMSE of 0.50 N s/m against the truth, less than half the exponential law's 1.10. Below about 0.2 m/s the network's curve is too high and peaks in the wrong place, which again traces back to the weakly identified static friction: c_{crit} depends on the slope of μ, and differentiating a learned curve amplifies its errors.

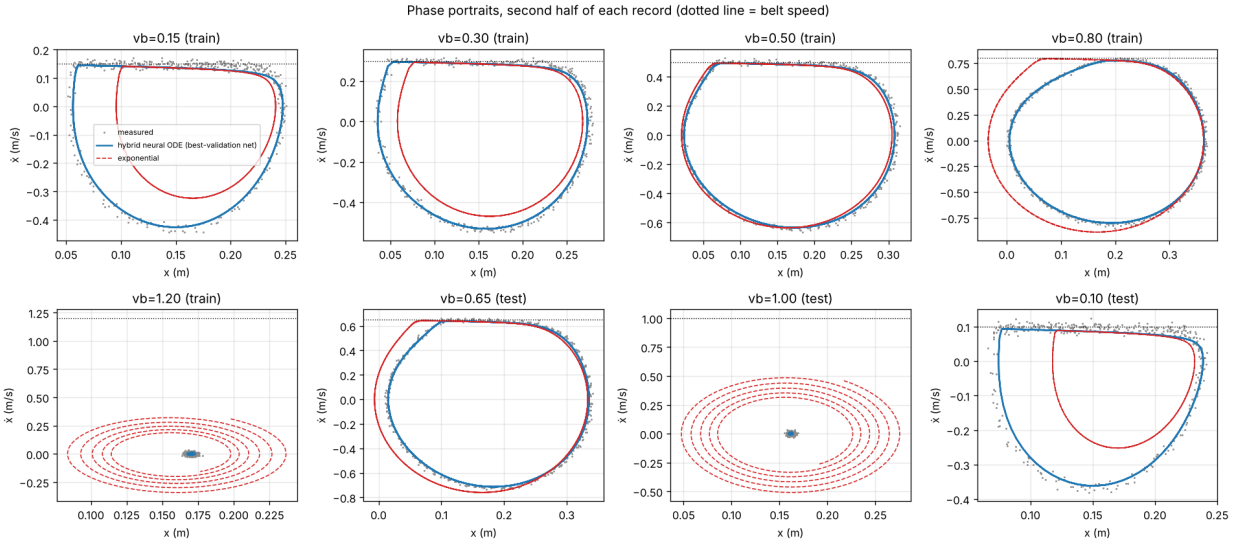


Figure 14: Phase portraits over the second half of each record for the best-validation network (blue), the exponential law (red) and the measurements (grey). The flat top at the belt speed (dotted) is the stick phase. The exponential law's cycles are too small at low speed and persist at 1.0 m/s where the truth settles (at 1.2 m/s they are still decaying at 15 s).

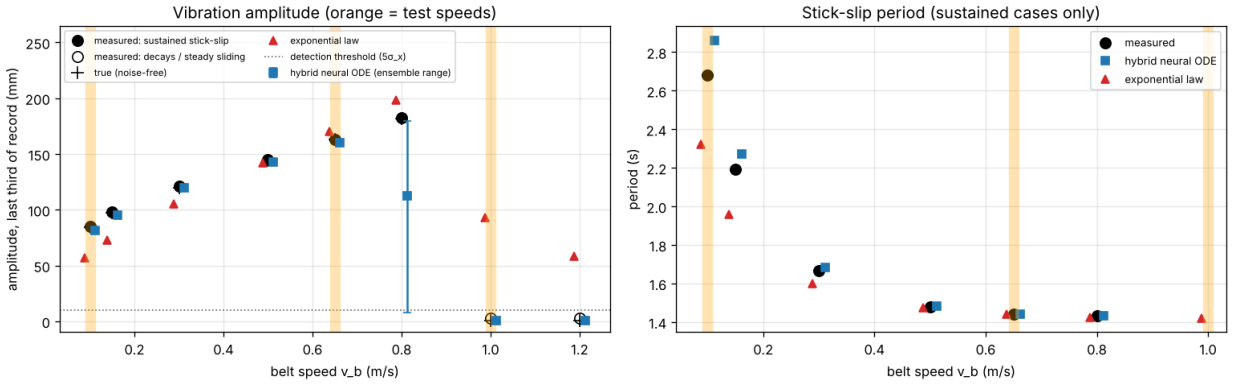


Figure 15: Left: vibration amplitude against belt speed; filled circles are sustained stick-slip, open circles decay to sliding, + marks the noise-free truth, the bar is the range over the five networks and the dotted line is the detection threshold ($5\sigma_x = 10\,\mathrm{mm}$). Right: stick-slip period for sustained cases. Orange bands are test speeds.

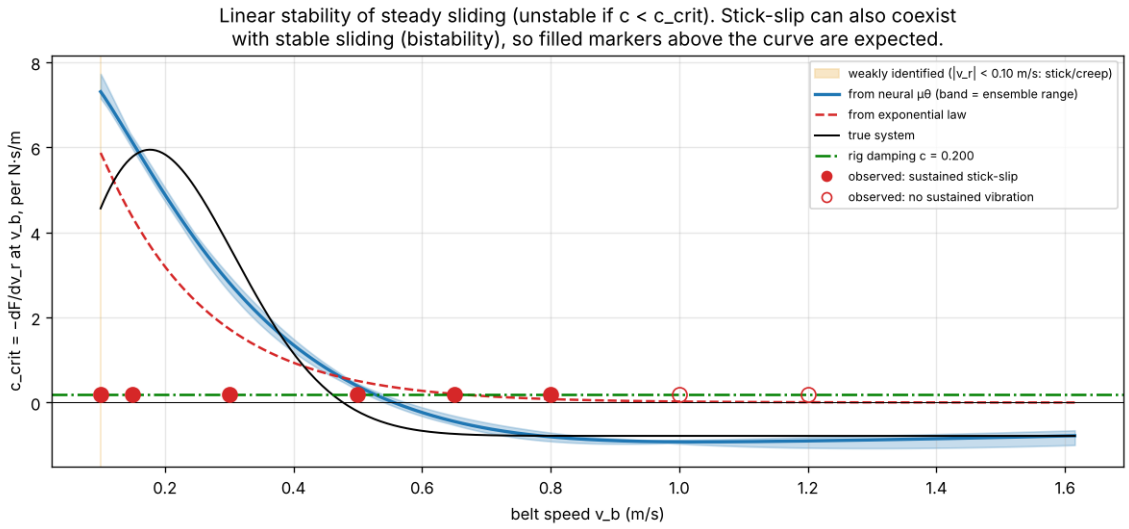


Figure 16: Linear stability limit $c_{\mathrm{crit}} = -\mathrm{d}F/\mathrm{d}v_r$ at v_b, computed from each friction model including the tanh regularisation, against the rig damping $c = 0.2\,\mathrm{N\,s/m}$ (green). Red markers show the observed regime at each speed.

The figure also shows bistability directly. At 0.5, 0.65 and 0.8 m/s the true c_{crit} is below the rig damping (-0.21, -0.74 and $-0.78\,\mathrm{N\,s/m}$), so steady sliding is linearly stable, yet the system released from rest settles on stick-slip. This is the same coexistence of states found in Section 5, now in the "experimental" data.

8.6 Residuals

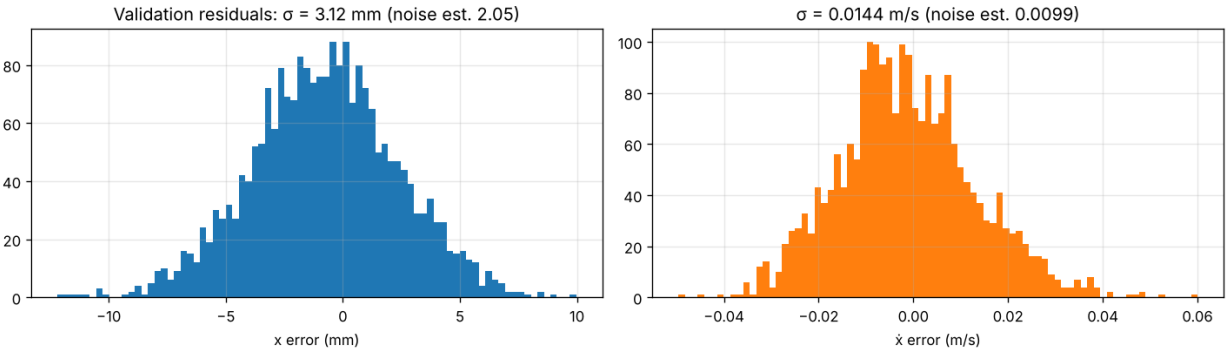


Figure 17: Errors of network 1 (validation loss 4.60, practically tied with the best) on the held-out validation windows: $\sigma = 3.12\,\mathrm{mm}$ in x and $0.0144\,\mathrm{m/s}$ in $\dot{x}$, against measurement noise of 2.05 mm and $0.0099\,\mathrm{m/s}$.

The residuals are centred and roughly Gaussian with no visible bias. They are wider than the noise for the same reason the noise floor exceeds 2: every window starts from a noisy state, and that error grows through the window.

9 Verification, limitations and discussion

The code was checked against independent references and audited by a separate reviewer before the final runs. That audit found ten problems in the first version (Table 11); all were fixed before the results above were produced.

Checks performed.

- *Solver:* RK4 against SciPy Radau, relative error 1.1×10^{-5} (Section 3). The reviewer repeated this at the corners of the Section 5 parameter box, where the error stays at or below 1.1×10^{-4}.
- *Gradients:* automatic-differentiation gradients of the expected vibration agree with central finite differences within about 7%, consistent with the sampling noise of the expectation (reviewer's check).
- *Physics:* signs, the linearised stability limit and the worst-case value of c_{crit} over the uncertainty box (2.907 N s/m) were checked by hand.
- *Labels and bistability:* classifier labels change in only 2 of 1 000 test cases when runs are doubled from 200 to 400 s, and all 29 apparent bistable cases were confirmed by restarting next to steady sliding.
- *Friction discovery:* the demo uses a known hidden law, a sharper friction switch than the model (no "inverse crime") and white noise of known size; the noise estimator recovers that size within 3%.
- *Numbers:* every result in Sections 3–8 is copied from the printed output or figures of the two programs; Section 2 comes from a reproduction of the 2024 code. An independent check of about 150 numbers in this report found none wrong.

Table 11: Problems found in review of the first version, and their fixes.

Problem in the first version	Consequence	Fix
Validation windows drawn at random from the same records as the training windows	"Validation" loss was in-sample	Last 15% of each record held out in time
Ensemble $\pm 2\sigma$ band presented as uncertainty	Truth often outside the band	Relabelled as spread; block bootstrap; coverage measured (52%)
Noise estimate from a filter residual	28% too low	Corrected by $1/\sqrt{1-h_0}$
Amplitude and period taken from noisy, partly decaying signals	Noise read as a 0.16 s "period"; transients read as limit cycles	Smoothing, a sustained-vibration test, hysteresis crossings
Stability plot ignored the tanh term	Wrong boundary at low speed	Exact derivative of the simulated force
Demo data made by the same model that was fitted	Optimistic accuracy	Truth uses a four-times-sharper friction switch
Classifier labels from 60 s runs	Slow transients labelled as stick-slip	200 s labels, checked against 400 s
Stiffness k optimised although J does not depend on it	Arbitrary k reported	Damping c is the only design variable
FNO error and speed quoted favourably	Static offset hid the vibration error; speed-up overstated	Error of the vibration part; measured timing against the simulator
Exponential baseline given fewer iterations	Unfair comparison	Same budget as each network

Limitations that remain.

- *Synthetic data only.* No measured rig data have been used yet; real data bring sensor offsets, drift, temperature effects and contact-stiffness dynamics that the demo lacks.

- *Model form.* Friction depends only on the slip speed: there is no dependence on normal load, temperature or memory (as in LuGre [4] or rate-and-state models), and the system has one degree of freedom, while brake squeal usually involves coupling between two or more modes.
- *Static friction and low-speed stability* are weakly identified (Sections 8.2 and 8.5).
- *Uncertainty.* The ensemble spread is not a calibrated confidence interval.
- *Separating c and μ.* When c is learned together with μ, damping and the viscous slope of μ are nearly interchangeable; c should be measured separately.
- *Smoothing parameter.* The tanh regularisation ($\varepsilon = 0.02\,\mathrm{m/s}$) changes vibration levels inside the stick-slip region by roughly 10–25% compared with $\varepsilon = 0.002\,\mathrm{m/s}$ (reviewer's check), although the regime boundary and the design result change little (J 0.1178 against 0.1197).
- *FNO benefit.* For this 1-DOF model the neural operator is barely faster than the batched simulator; its case rests on more expensive models.

What the results do and do not show. Deep learning helps most where it is combined with the physics: exact gradients from a differentiable simulator, a surrogate trained on that simulator, and a hybrid model that learns only the unknown friction law. The results do not yet show performance on real measurements, and the friction-discovery accuracy figures hold for this synthetic rig and noise level.

10 Conclusions and future work

Deep learning pays off on this problem when it works with the physics rather than replacing it.

1. **The 2024 code was not usable.** Its friction term injects energy, there is no belt to drive stick-slip, and its finite-difference gradients are solver noise, two to five orders of magnitude larger than the true gradient.
2. **A differentiable simulator makes true SGD possible.** Optimising damping under uncertain operating conditions gives $c = 2.09\,\mathrm{N\,s/m}$ and cuts sustained stick-slip on unseen conditions from 93.3% to 3.0%.
3. **An MLP surrogate predicts vibration with $R^2 = 0.999$ and the regime with 99.2% accuracy.** It reveals a bistable band above the linear stability limit (29 confirmed test cases), so designs based on linear stability alone are non-conservative. Optimising through it gives the same design within 0.1% in J, about 120 times faster.
4. **A Fourier neural operator maps a time-varying belt speed to the full response** with a median velocity error of 6.8%. It struggles where the response switches regime, and for a 1-DOF model it saves little time.
5. **A hybrid neural ODE discovers an unknown friction law from noisy vibration records.** Its RMSE of 0.0041 in μ is seven times better than the classical exponential law, and it alone predicts the correct regime at an unseen speed. Static friction and calibrated uncertainty remain open.

Future work.

- Apply `friction_law_discovery.py` to pin-on-disc or brake-rig records, with c and k from separate modal or free-decay tests.
- Extend the learned law to depend on normal load and temperature, and add a friction state with memory (LuGre or rate-and-state) as a hybrid model.

- Calibrate uncertainty with more diverse data, likelihood-based ensembles or Bayesian networks, and check coverage again on the demo.
- Move to a two-degree-of-freedom mode-coupling model of brake squeal, where the surrogate and neural-operator approaches carry over and the operator's speed advantage becomes real.

A Reproducing the results on a PyTorch PC

Both programs need Python 3.9 or newer with PyTorch, NumPy, SciPy and Matplotlib; Jupyter is needed for the notebook. They run on a CPU; a GPU helps little because the tensors are small and the time loop is sequential.

Table 12: Programs and run times.

File	Produces	CPU time
`reproduce_original_flaws.py`	Section 2: Fig. 1	about 2 min
`friction_vibration_dl.ipynb`	Sections 3–6: four figures and all printed metrics	about 25 min
`friction_law_discovery.py -demo`	Sections 7–8: 11 plots, `summary.json`, `learned_friction_curve.csv`, `models.pt`	about 25 min

```
# notebook, executed in place with all outputs saved
jupyter nbconvert --to notebook --execute --ExecutePreprocessor.timeout=10800 \
       friction_vibration_dl.ipynb --output friction_vibration_dl.ipynb

# friction-law discovery on the synthetic demo (writes ./friction_outputs)
python friction_law_discovery.py --demo
python friction_law_discovery.py --demo --quick # 2 networks, 400 iterations
python friction_law_discovery.py --demo --load friction_outputs/models.pt # replot only

# your own rig data, with c and k from a separate test
python friction_law_discovery.py --data my_rig --m 1.0 --k 20 --c 0.2
```

Results on another machine can differ in the last digits, because random-number streams and floating-point summation order vary between PyTorch versions and between CPU and GPU.

Data format for real measurements. The folder passed to `-data` holds `experiments.csv`, one row per test, plus one CSV per test with columns `t,x` (and `v` if measured; otherwise it is computed from x with a Savitzky–Golay derivative, which is noisier). SI units throughout.

```
file,vb,N,role
run_015.csv,0.15,9.81,train
run_030.csv,0.30,9.81,train
run_050.csv,0.50,9.81,train
run_065.csv,0.65,9.81,test
```

Practical advice for rig data:

- give `vb` as the sliding speed at the contact; for a disc, ωr at the pin;
- use at least four or five belt speeds that span both stick-slip and steady sliding, and hold one or two out as `test`;
- measure c separately (free decay with the contact lifted); with `-learn-ck`, c and the viscous slope of μ are nearly interchangeable and c is pinned only by the mean position offset;

- if the validation loss stays far above the training loss, or the ensemble spread is wide, more belt speeds or longer records help more than more iterations.

References

[1] V. Kumar, *Deep Learning for Friction-Induced Vibration*, GitHub repository, `https://github.com/vikashsonidaudnagar/deep-learning-friction-induced-vibration`. The first version of this project (August 2024, originally titled *Optimization of Friction-Induced Vibration System Using Stochastic Gradient Descent*) is kept in its `legacy_2024` folder.

[2] R. A. Ibrahim, Friction-induced vibration, chatter, squeal, and chaos, Parts I and II, *Applied Mechanics Reviews* 47(7), 209–253, 1994.

[3] E. J. Berger, Friction modeling for dynamic system simulation, *Applied Mechanics Reviews* 55(6), 535–577, 2002.

[4] C. Canudas de Wit, H. Olsson, K. J. Åström, P. Lischinsky, A new model for control of systems with friction, *IEEE Transactions on Automatic Control* 40(3), 419–425, 1995.

[5] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, Neural ordinary differential equations, *NeurIPS*, 2018. arXiv:1806.07366.

[6] C. Rackauckas et al., Universal differential equations for scientific machine learning, 2020. arXiv:2001.04385.

[7] E. M. Turan, J. Jäschke, Multiple shooting for training neural differential equations on time series, *IEEE Control Systems Letters*, 2022, doi:10.1109/LCSYS.2021.3135835. arXiv:2109.06786.

[8] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, A. Anandkumar, Fourier neural operator for parametric partial differential equations, *ICLR*, 2021. arXiv:2010.08895.

[9] B. Lakshminarayanan, A. Pritzel, C. Blundell, Simple and scalable predictive uncertainty estimation using deep ensembles, *NeurIPS*, 2017. arXiv:1612.01474.

[10] A. Savitzky, M. J. E. Golay, Smoothing and differentiation of data by simplified least squares procedures, *Analytical Chemistry* 36(8), 1627–1639, 1964.

[11] A. Paszke et al., PyTorch: an imperative style, high-performance deep learning library, *NeurIPS*, 2019. arXiv:1912.01703.